Product Requirements Document (PRD): Grama-Suvidha

1. Executive Summary

Product Name: Grama-Suvidha
Platform: Android Mobile Application
Primary Language: Kotlin (Jetpack Compose)

Overview: Grama-Suvidha is a transparency-focused mobile application built to empower rural citizens by giving them direct visibility into government-funded infrastructure projects. By providing localized data, offline support, and interactive feedback mechanisms, Grama-Suvidha bridges the communication gap between citizens and local authorities, fostering a culture of accountability and community-driven development.

---

2. Problem Statement

In many rural areas, there is a significant lack of transparency and accessibility regarding government-funded infrastructure projects. Citizens often struggle to stay informed about the progress of local development, understand how public funds are allocated, or report issues directly to the authorities. Existing platforms are rarely optimized for rural demographics, often lacking native language support (such as Kannada) and robust offline accessibility. This disconnect breeds mistrust and prevents the community from actively participating in their village's growth.

---

3. Product Vision & Goals

Vision

To create a fully transparent, accessible, and inclusive digital ecosystem where every villager can track, verify, and contribute to the infrastructural development of their community.

Key Goals

1. **Promote Transparency:** Ensure all project budgets, timelines, and visual proof of progress are publicly accessible.
2. **Inclusivity via Localization:** Break language barriers by offering first-class support for regional languages (starting with Kannada).
3. **Resilient Access:** Guarantee application usability even in areas with poor or no internet connectivity through robust offline caching.
4. **Actionable Feedback:** Provide a streamlined channel for citizens to rate projects and report infrastructural issues.

---

4. Target Audience & User Personas

Persona 1: The Engaged Villager (Ramesh, 45, Farmer)

- **Background:** Lives in the village, speaks only Kannada, has a basic smartphone with a limited data plan.

- **Pain Points:** Hears about road repairs but never sees the budget or timeline. Doesn't know who to complain to if a road is left half-finished.
- **Needs:** A simple interface in Kannada that works offline, showing him exactly what is being built and allowing him to report issues if the work stops.

Persona 2: The Local Authority / Panchayat Member (Sunita, 38, Administrator)

- **Background:** Manages local funds and oversees project execution. Needs to show progress to the state government and the citizens.
- **Pain Points:** Faces complaints about lack of transparency. Struggles to visually demonstrate the impact of completed projects.
- **Needs:** A platform where citizens can see the "Before & After" transformations, reducing manual queries and building trust.

---

5. User Stories & Acceptance Criteria

Epics

1. **Localization & Accessibility**
    - *User Story:* As a user, I want to toggle the app language between English and Kannada so that I can understand the content in my native tongue.
    - *Acceptance Criteria:* A dedicated language toggle exists. All UI elements, project titles, descriptions, and statuses translate instantly without requiring a restart.
2. **Project Discovery & Tracking**
    - *User Story:* As a user, I want to view a list of ongoing and completed projects with their budget and timeline so I know how public funds are spent.
    - *Acceptance Criteria:* The dashboard displays a scrollable list of projects fetched from the data source. Each card shows the title, budget, date, and a visual progress bar.
3. **Visual Verification**
    - *User Story:* As a user, I want to see the physical progress of a project through "Before and After" images so I can verify the work.
    - *Acceptance Criteria:* The project detail screen includes an interactive image slider allowing the user to seamlessly swipe between the 'Before' and 'After' states of the infrastructure.
4. **Citizen Feedback System**
    - *User Story:* As a user, I want to rate a project and submit a textual report so that authorities are aware of my satisfaction or any pending issues.
    - *Acceptance Criteria:* A feedback block is available on the project detail screen with a 1-5 star rating component and a text input field for issue reporting.
5. **Offline Capability**
    - *User Story:* As a user, I want to view project details even when my internet connection drops while I am out in the village.
    - *Acceptance Criteria:* The application caches the JSON data locally. Upon subsequent app launches without an internet connection, the cached data is parsed and displayed instantly.

---

6. Functional Requirements

6.1 Dashboard (Home Screen)

- Must display a dynamic list of projects using Jetpack Compose `LazyColumn`.
- Must show a progress indicator (0-100%) mapped to the underlying database state.
- Must include a global Language Toggle button (EN / KN) in the Top App Bar.

6.2 Project Details Screen

- Must parse and display detailed project fields: `title`, `budget`, `completionDate`, `progress`, `status`, and `description`.
- Must render an interactive "Before & After" image slider. Images must load asynchronously using the Coil library.
- Must include the Citizen Feedback form.

6.3 Data Management

- The application must simulate network fetching using a Mock API (parsing `assets/projects.json`).
- Must utilize `StateFlow` within the ViewModel to manage and emit UI states (Loading, Success, Error).

---

7. Non-Functional Requirements (NFRs)

7.1 Performance

- The UI must render at a consistent 60fps. Jetpack Compose recompositions must be optimized to prevent UI stuttering, especially when dragging the image slider.
- Image loading must be highly optimized with memory and disk caching (managed by Coil) to prevent `OutOfMemory` errors on low-end devices.

7.2 Usability & UX

- The application must adhere to Material Design 3 guidelines.
- Must employ "Glassmorphic" premium design elements to ensure a modern, trustworthy feel.
- Touch targets for all interactive elements (buttons, sliders) must be at least 48x48dp to accommodate users who may not be highly digitally literate.

7.3 Reliability & Offline Support

- The app must never crash due to a network failure.
- Offline viewing must be instant after the initial fetch. The repository layer must handle the fallback to local cache seamlessly without exposing the error to the UI layer.

---

8. Technical Architecture & Constraints

8.1 Tech Stack

- **Language:** Kotlin
- **UI Toolkit:** Jetpack Compose
- **Architecture:** MVVM (Model-View-ViewModel)
- **Reactive Programming:** Kotlin Coroutines & `StateFlow`
- **Image Loading:** Coil
- **Serialization:** `kotlinx.serialization`

8.2 Data Schema (Mock API)

The application relies on a strictly defined JSON contract. Example:

```
{
  "id": 1,
  "titleEn": "Main Road Repair",
  "titleKn": "ಮುಖ್ಯ ರಸ್ತೆ ದುರಸ್ತಿ",
  "budget": "₹5,00,000",
  "completionDate": "2024-05-15",
  "progress": 75,
  "statusEn": "In Progress",
  "statusKn": "ಪ್ರಗತಿಯಲ್ಲಿದೆ",
  "beforeImageUrl": "file:///android_asset/road_before.jpg",
  "afterImageUrl": "file:///android_asset/road_after.jpg",
  "descriptionEn": "Repairing the main approach road...",
  "descriptionKn": "ಗ್ರಾಮದ ಕೇಂದ್ರಕ್ಕೆ ಹೋಗುವ..."
}
```

---

9. Future Roadmap (Post-Internship)

1. **Backend Integration:** Replace the Mock JSON API with a live REST/GraphQL API hosted on AWS/Firebase.
2. **Authentication:** Implement OTP-based mobile login for citizens to track their specific complaints.
3. **Geotagging:** Require government contractors to upload "After" images with embedded GPS metadata to prevent fraud.
4. **Push Notifications:** Alert subscribed villagers when a project in their ward begins or finishes.